Heart Disease Prediction Using Logistic Regression and Random Forest: A Machine Learning Approach

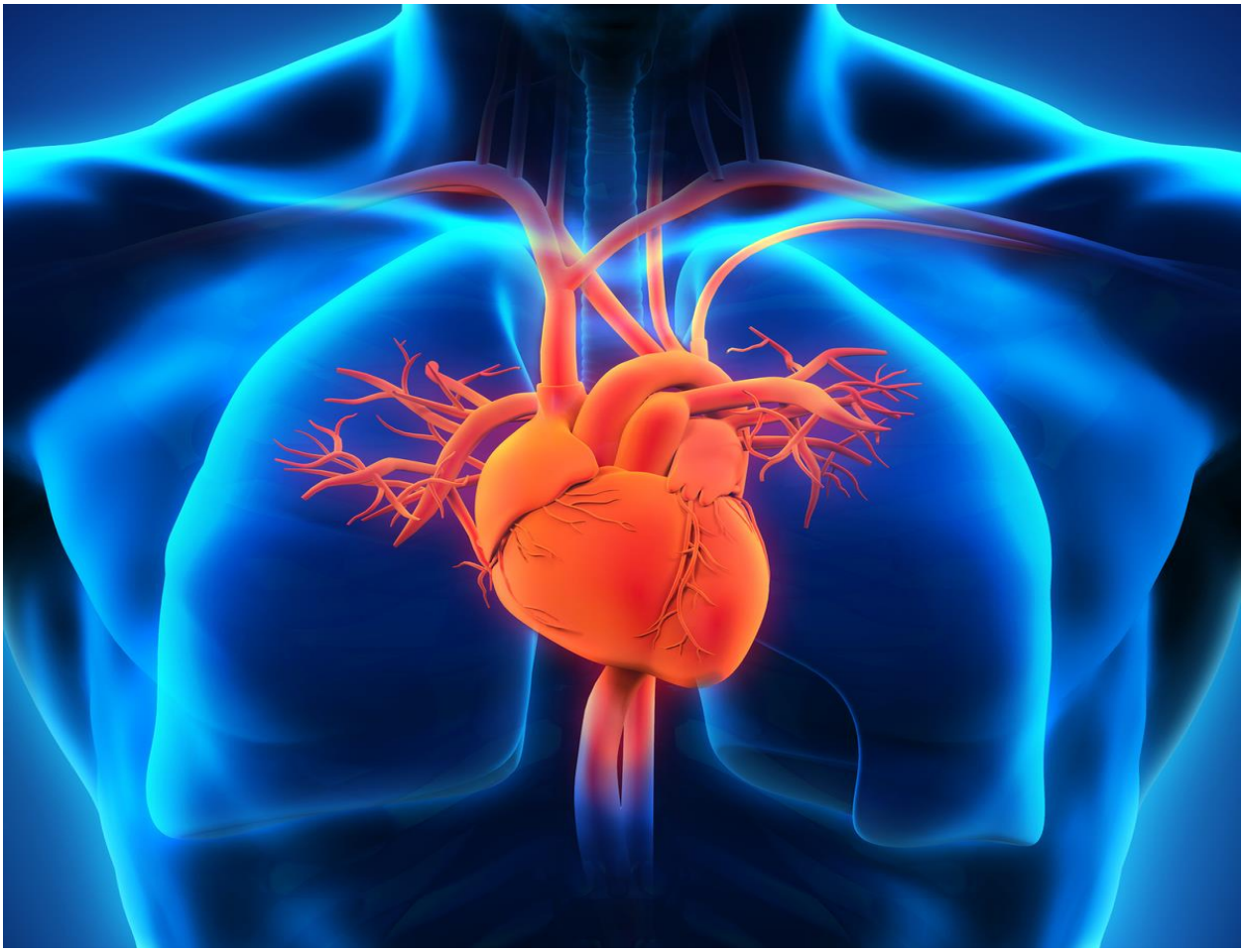
PROJECT REPORT

CSC – 5800 INTELLIGENT SYSTEMS

DEPARTMENT OF COMPUTER SCIENCE

WAYNE STATE UNIVERSITY – APRIL 2025

SUBMITTED BY – POORNESH MUPPALAM



1. Overview of the Problem: Heart disease remains one of the leading causes of mortality globally. Leveraging patient health records for early prediction can enable timely medical intervention and potentially save lives. In this project, we aim to predict the presence and severity of heart disease using a structured dataset containing patient metrics such as age, sex, chest pain type, fasting blood sugar, resting blood pressure, cholesterol levels, electrocardiogram results, exercise-induced angina, and maximum heart rate achieved.

2. LITERATURE SURVEY:

Predicting the presence of coronary artery disease (CAD) has long been a critical research area in medical diagnostics and clinical decision support systems. Numerous studies have attempted to model CAD using various statistical and machine learning techniques. The Cleveland Heart Disease dataset, curated by the UCI Machine Learning Repository, is one of the most frequently used datasets for these analyses. In this section, we explore three foundational studies that contributed significantly to the domain.

2.1 Detrano et al. (1989): Probability Algorithms for CAD Diagnosis

In their landmark study, **Detrano et al. (1989)** introduced a probability-based diagnostic algorithm aimed at the **international application of coronary artery disease prediction**. Their work utilized logistic regression models trained on patient clinical features, including age, chest pain type, blood pressure, cholesterol, and electrocardiographic results. This study emphasized the **development of a reproducible scoring system** that could generalize across populations in different countries.

Key contributions:

- Provided an early **probabilistic modeling approach** for disease diagnosis.
- Showed that **clinical variables could be effectively modeled** to approximate the likelihood of CAD presence.
- Validated the model's performance across international datasets, stressing its **global applicability**.
- Formed the foundational structure for the dataset now widely referred to as the **Cleveland Heart Disease dataset**.

Their model highlighted that traditional regression-based methods could form reliable baselines for future comparative machine learning models.

2.2 Aha & Kibler (1991): Instance-Based Learning with the Cleveland Dataset

In their study titled *"Instance-based prediction of heart-disease presence with the Cleveland database"*, **David Aha and Dennis Kibler** explored the use of **instance-based learning (IBL)** approaches, particularly focusing on the **k-nearest neighbor (k-NN)** algorithm. Unlike global models like regression, IBL makes predictions using the **similarity between test cases and historical examples**, thereby offering a **local generalization strategy**.

Key contributions:

- Demonstrated the **effectiveness of lazy learning techniques** such as k-NN in medical diagnosis.
- Highlighted that **no prior model building** is needed, making it suitable for dynamically changing datasets.
- Discussed the importance of **feature scaling and distance metrics** in improving classification performance.
- Provided comparative benchmarks against traditional classifiers like decision trees and linear models.

Their work showed that instance-based methods could serve as strong non-parametric alternatives for heart disease prediction, especially when interpretability and simplicity are prioritized.

2.3 Gennari, Langley & Fisher (1989): Incremental Concept Formation

The work by **Gennari, Langley, and Fisher** in *"Models of incremental concept formation"* introduced the **COBWEB algorithm**, an incremental clustering approach designed for **hierarchical concept learning**. Though not focused specifically on the Cleveland dataset, the principles they proposed are highly relevant for **continuous learning and medical knowledge representation**.

Key contributions:

- Proposed an **incremental learning model** where new patient data can be assimilated without retraining from scratch.
- COBWEB forms **hierarchical clusters** based on attribute distributions, which is useful for understanding **multi-level disease profiles**.
- Emphasized **concept drift adaptation**, a significant advantage in medical settings where disease manifestation may evolve over time.

Their study is particularly valuable in modern-day applications involving **online learning systems** and **real-time patient monitoring**, which are becoming increasingly relevant with the rise of wearable devices and electronic health records.

3. Dataset Overview: The dataset used is based on the refined version of UCI Heart Disease dataset compiled from multiple medical sources and made publicly accessible via **Kaggle.** It contains **920 patient records**, each detailing various health-related metrics used to predict the presence of heart disease.
Features: This includes 14 attributes, including

- **Demographic Attributes**:
  - `age`: Age of the patient in years
  - `sex`: Sex of the patient (1 = male; 0 = female)
- **Clinical Measurements**:
  - `trestbps`: Resting blood pressure (in mm Hg)
  - `chol`: Serum cholesterol level (in mg/dl)
  - `thalach`: Maximum heart rate achieved
- **Categorical/Diagnostic Attributes**:
  - `cp`: Type of chest pain (4 categories)
  - `restecg`: Resting electrocardiographic results
  - `exang`: Exercise-induced angina (1 = yes; 0 = no)

4. Data Cleaning and Preprocessing:

To prepare the dataset for effective machine learning modeling, a systematic data cleaning and preprocessing pipeline was applied. These steps were essential for ensuring data quality, handling inconsistencies, and optimizing model performance.

1. Handling Missing Values: The original UCI Heart Disease dataset included certain features with a significant number of missing or ambiguous values. In particular:

> **`ca`**: Indicates the number of major vessels colored by fluoroscopy.
> **`thal`**: Represents thalassemia (a blood disorder) and includes values like 3, 6, and 7, which were inconsistently represented or missing in many cases.

Due to the **high proportion of missing or unprocessable entries**, these two attributes were **removed** to maintain the dataset's integrity and avoid biased imputations that could distort predictions.

2. **Removal of Redundant Metadata:** Columns such as:

- **`id`**: A unique identifier for each patient record.
- **`dataset`**: An identifier indicating the source of the patient record (e.g., Cleveland, Hungarian, etc.)

These fields were **dropped** as they serve no predictive purpose and can introduce noise or model overfitting if left in the feature set.

3. **Target Variable Conversion:**

Target Variable – num is originally a multiclass attribute indicating the severity of heart disease, this variable has been **converted into a binary classification.**
0 – No heart disease
1 – Presence of heart disease

This transformation simplifies the problem into a **binary classification task**, which is more interpretable for both medical diagnosis and machine learning performance evaluation.

4. Categorical Encoding

Some attributes, such as `cp` (chest pain type), `restecg` (resting ECG), and `exang` (exercise-induced angina), are **categorical** in nature but represented numerically. These variables were:

- **Retained as categorical integers**
- Optionally subjected to **one-hot encoding** (in some experiments), especially when used with algorithms sensitive to ordinal relationships (e.g., Logistic Regression)

5. Feature Scaling

Numerical features such as:

- `age`, `trestbps`, `chol`, `thalach`, `oldpeak`

were **standardized using `StandardScaler`** from Scikit-learn to ensure uniform scaling. This is particularly important for algorithms such as:

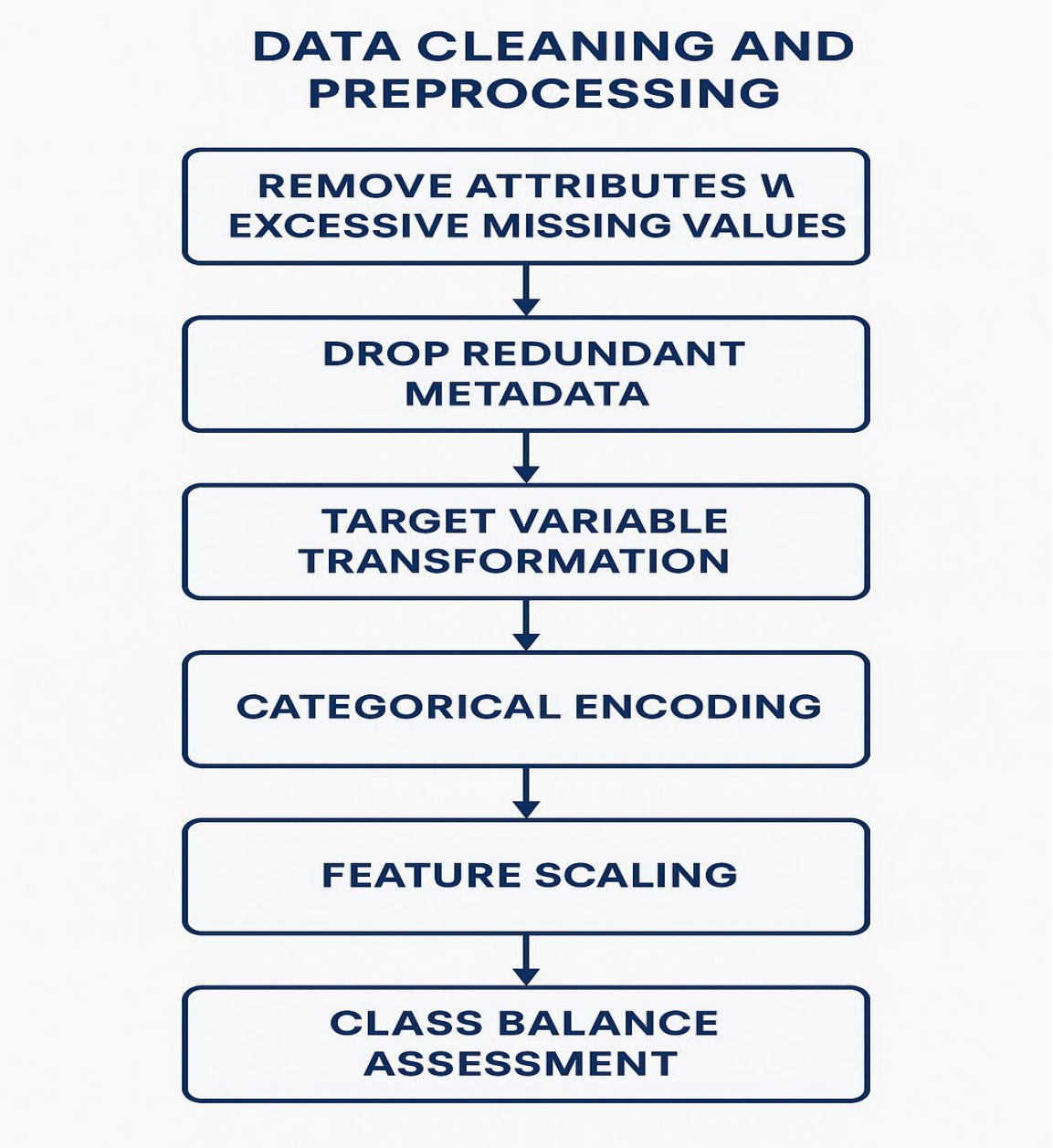
- Logistic Regression
- Support Vector Machines (SVMs)
- K-Nearest Neighbors (KNN)

which are sensitive to the scale of input features.

6. Class Balance Check

The distribution of the target variable was evaluated to identify any **class imbalance**. While mild imbalance was present (slightly more patients with disease), it was not severe enough to warrant the use of oversampling techniques such as SMOTE. However, performance metrics such as **precision, recall, and F1-score** were carefully monitored during evaluation.

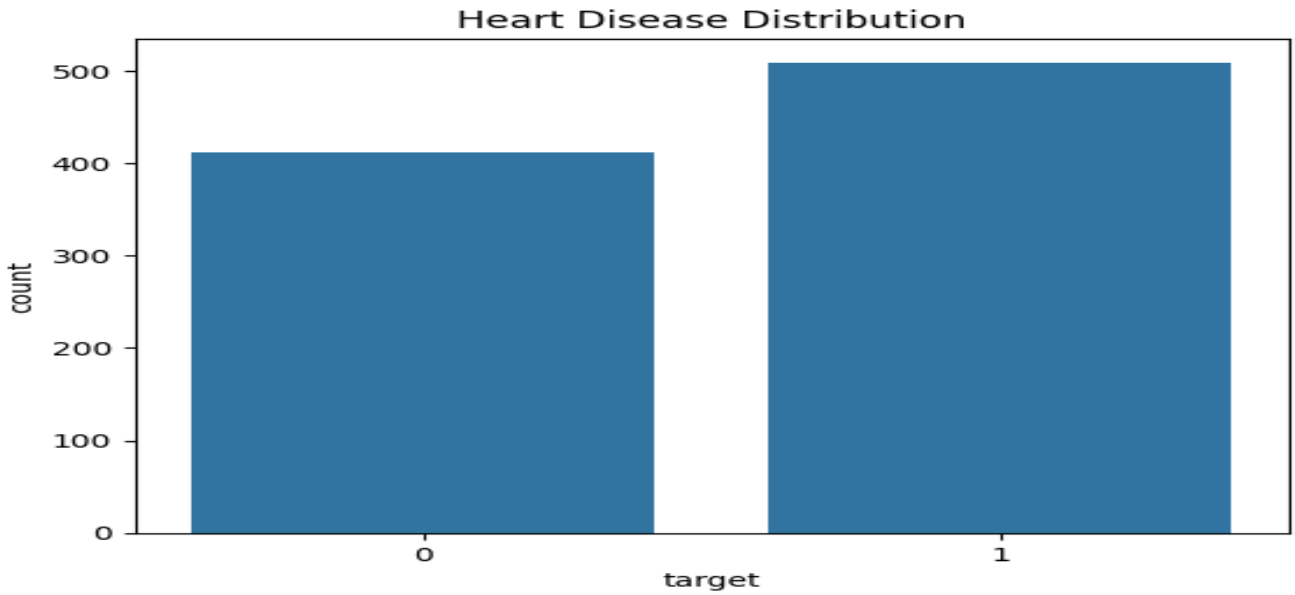
Flowchart of Data cleaning and Pre-processing:



5. Visualization: Visualizing the dataset provides critical initial insights into the underlying distribution of features and their potential relationship with heart disease. The following plots were generated to explore the structure and separability of the data.

Countplot of Target Variable

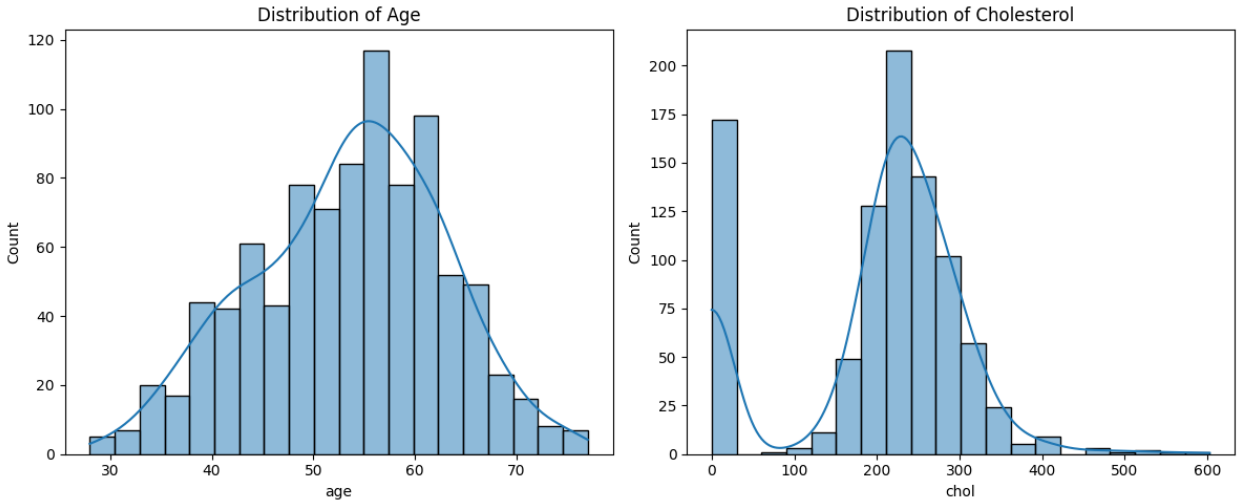
Purpose: To assess the class distribution of the target (`num`)



Observation:
The countplot revealed a **slightly higher number of patients with heart disease (target = 1)** compared to those without (target = 0).

Histograms

- **Features Visualized**: `age` and `chol` (serum cholesterol)
- **Purpose**: To examine the spread and central tendency of these features across all patients.

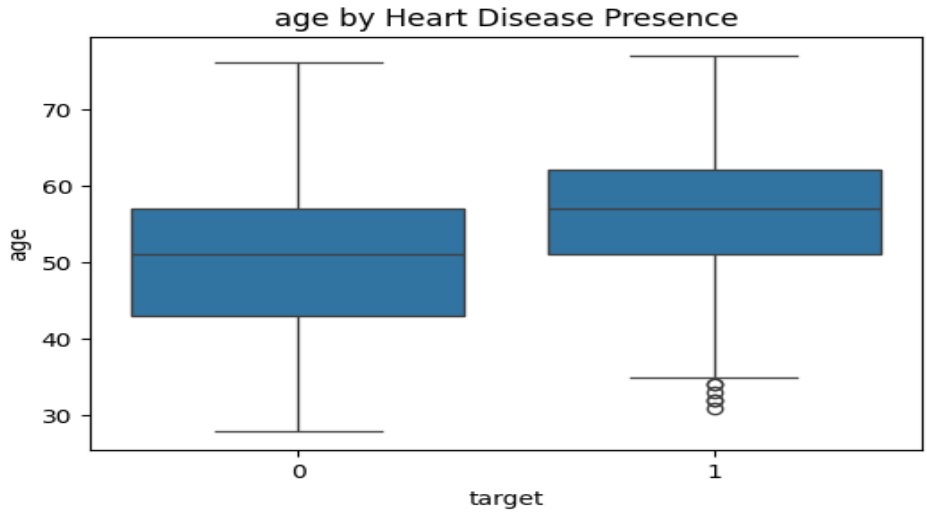


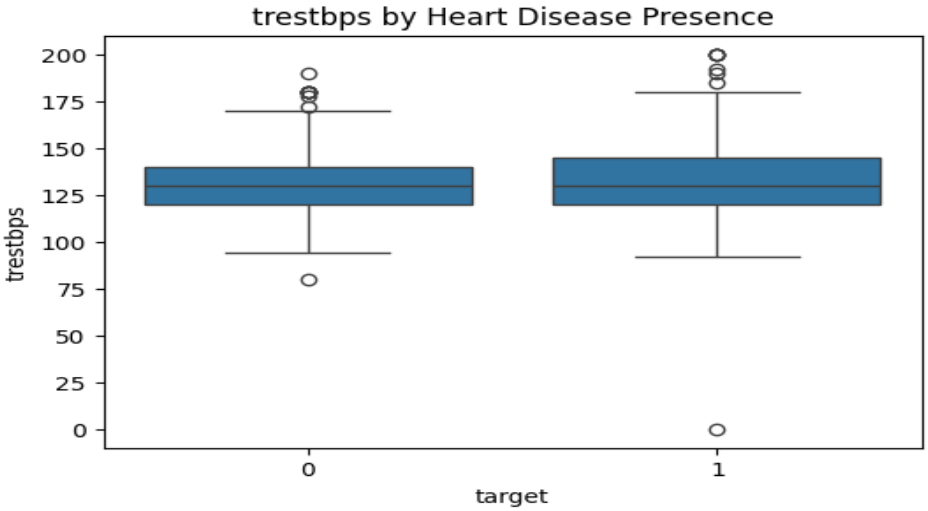
- **Observation**:
  - `age` showed a fairly normal distribution centered around middle age (~50-60 years).

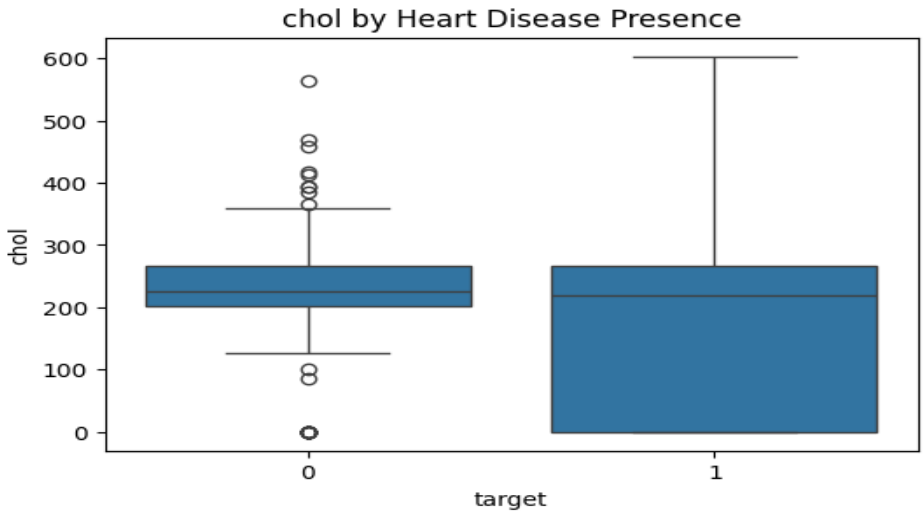
- `chol` exhibited **positive skewness**, with most values clustering around 200–250 mg/dl but a few outliers on the higher end.
- Neither `age` nor `chol` appeared to distinctly separate patients with or without heart disease, indicating limited discriminative power on their own.

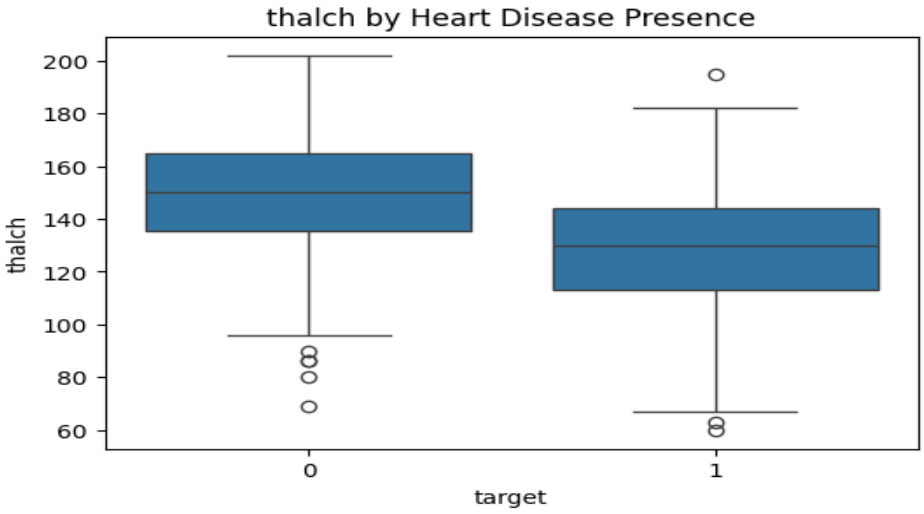
Boxplots

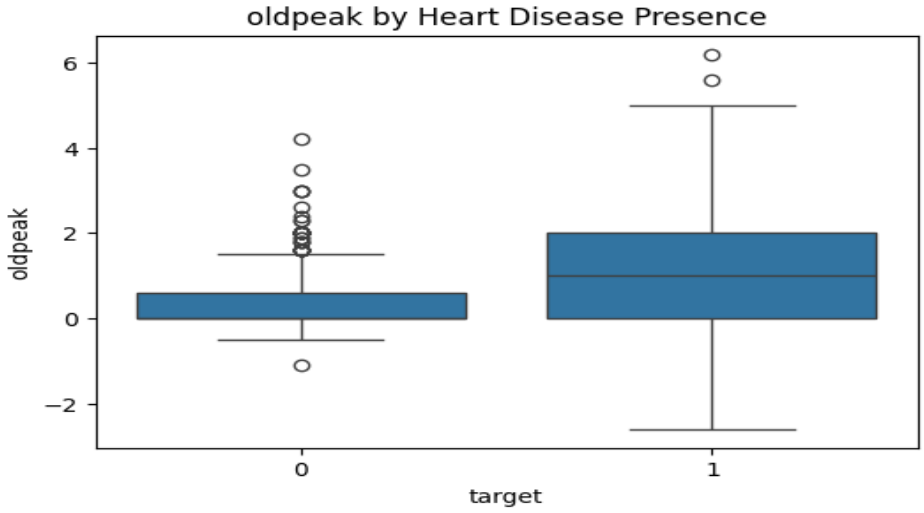
- **Features Compared**: `oldpeak`, `thalach`, `chol`, `age`, `trestbps`
- **Purpose**: To identify how numeric features differ between target classes (0 vs 1).






chol by Heart Disease Presence
chol
600
500
400
300
200
100
0
0
1
target

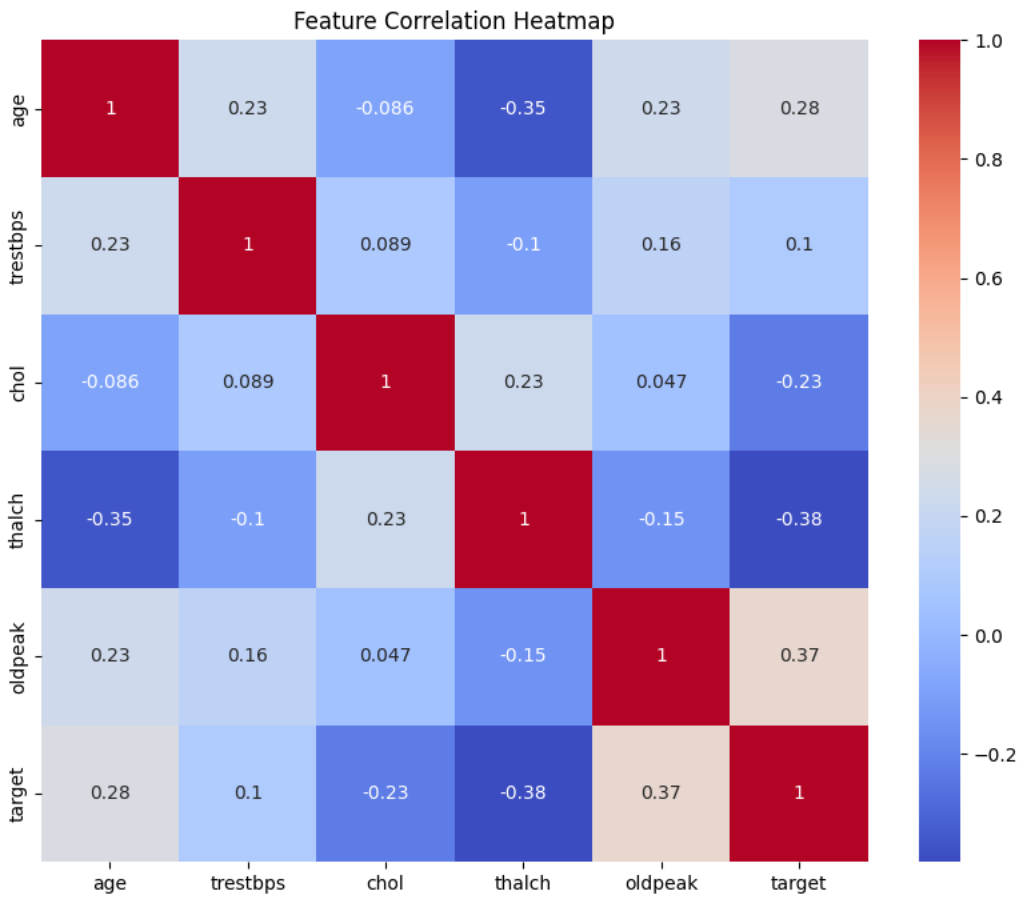

thalch by Heart Disease Presence
thalch
200
180
160
140
120
100
80
60
0
1
target


oldpeak by Heart Disease Presence
oldpeak
6
4
2
0
−2
0
1
target

- **Key Insights**:
  - `oldpeak` (ST depression induced by exercise) had **higher values among heart disease patients**, indicating a meaningful relationship.
  - `thalach` (maximum heart rate achieved) was generally **lower in patients with heart disease**, highlighting its potential as a predictive feature.
  - `age`, `chol`, and `trestbps` did not show strong separation between the two target classes, suggesting these features may require **combinatorial analysis or interaction modeling** for better impact.

Correlation Heatmap

- **Purpose**: To visualize Pearson correlations between all numeric features and identify redundancies or strong relationships.



- **Observation**:
  - `thalach` showed a **strong negative correlation with the target variable**, reinforcing its inverse relationship with heart disease presence.
  - Other moderate correlations were seen, but no multicollinearity issues were detected among the predictors.
  - The heatmap helped confirm that certain variables like `chol` and `age` are weakly correlated with heart disease, which aligns with earlier visual findings.

6. Algorithm Selection:

To predict the presence of heart disease, two supervised learning algorithms were selected for comparative evaluation: a **baseline linear model** and a **nonlinear ensemble model**.

Algorithm Used:

- **Logistic Regression** (baseline):

  Chosen as a **baseline** due to its simplicity, interpretability, and suitability for binary classification tasks.

  Captures **linear relationships** between features and the target variable.

- **Random Forest Classifier** (ensemble model):

  A powerful **ensemble learning model** that builds multiple decision trees and aggregates their outputs.

  Handles **nonlinear relationships and interactions** between features effectively.

  Provides **feature importance** for model interpretability.

```python
# Predictions on Train Set
y_train_pred_lr = lr.predict(X_train)
y_train_pred_rf = rf.predict(X_train)

from sklearn.metrics import classification_report, roc_auc_score

print("📘 Logistic Regression - Training Set")
print(classification_report(y_train, y_train_pred_lr))
print("Train AUC (LR):", roc_auc_score(y_train, lr.predict_proba(X_train)[:, 1]))

print("\n📗 Random Forest - Training Set")
print(classification_report(y_train, y_train_pred_rf))
print("Train AUC (RF):", roc_auc_score(y_train, rf.predict_proba(X_train)[:, 1]))
```

```
📘 Logistic Regression - Training Set
              precision    recall  f1-score   support

           0       0.81      0.78      0.79       336
           1       0.82      0.85      0.83       400

    accuracy                           0.82       736
   macro avg       0.81      0.81      0.81       736
weighted avg       0.82      0.82      0.81       736

Train AUC (LR): 0.8915252976190475

📗 Random Forest - Training Set
              precision    recall  f1-score   support

           0       0.84      0.81      0.83       336
           1       0.85      0.87      0.86       400

    accuracy                           0.84       736
   macro avg       0.84      0.84      0.84       736
weighted avg       0.84      0.84      0.84       736

Train AUC (RF): 0.9206436011904762
```

```
Train AUC (RF): 0.9206436011904762
```

```python
# Test set evaluation
print("\n📌 Logistic Regression - Test Set")
print(classification_report(y_test, y_pred_lr))
print("Test AUC (LR):", roc_auc_score(y_test, lr.predict_proba(X_test)[:, 1]))

print("\n📌 Random Forest - Test Set")
print(classification_report(y_test, y_pred_rf))
print("Test AUC (RF):", roc_auc_score(y_test, rf.predict_proba(X_test)[:, 1]))
```

```
📌 Logistic Regression - Test Set
              precision    recall  f1-score   support

           0       0.74      0.77      0.76        75
           1       0.84      0.82      0.83       109

    accuracy                           0.80       184
   macro avg       0.79      0.79      0.79       184
weighted avg       0.80      0.80      0.80       184

Test AUC (LR): 0.88782874617737

📌 Random Forest - Test Set
              precision    recall  f1-score   support

           0       0.84      0.84      0.84        75
           1       0.89      0.89      0.89       109

    accuracy                           0.87       184
   macro avg       0.86      0.86      0.86       184
weighted avg       0.87      0.87      0.87       184

Test AUC (RF): 0.899204892966361
```

Loss/Error Rate for Logistic and Random Forest:

```python
from sklearn.metrics import accuracy_score

# Compute Error Rate (1 - accuracy)
error_lr = 1 - accuracy_score(y_test, y_pred_lr)
error_rf = 1 - accuracy_score(y_test, y_pred_rf)

print("\n❌ Error / Loss Metrics:")
print(f"Logistic Regression Error Rate: {round(error_lr, 4)}")
print(f"Random Forest Error Rate: {round(error_rf, 4)}")
```

```
❌ Error / Loss Metrics:
Logistic Regression Error Rate: 0.2011
Random Forest Error Rate: 0.1304
```

Parameter Decisions:

- **Logistic Regression**
  - `max_iter = 1000` was set to ensure proper convergence, especially after feature scaling.
- **Random Forest (Tuned)**

- `max_depth = 5`: Limits the depth of trees to avoid overfitting
- `min_samples_leaf = 5`: Ensures minimum samples per leaf node for robustness
- `n_estimators = 100`: Number of trees in the ensemble for stable predictions

These hyperparameters were selected through **manual tuning and cross-validation** to balance model complexity and generalization.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report

print("Logistic Regression Report:")
print(classification_report(y_test, y_pred_lr))

print("Random Forest Report:")
print(classification_report(y_test, y_pred_rf))
```

```
Logistic Regression Report:
              precision    recall  f1-score   support

           0       0.74      0.77      0.76        75
           1       0.84      0.82      0.83       109

    accuracy                           0.80       184
   macro avg       0.79      0.79      0.79       184
weighted avg       0.80      0.80      0.80       184

Random Forest Report:
              precision    recall  f1-score   support

           0       0.84      0.84      0.84        75
           1       0.89      0.89      0.89       109

    accuracy                           0.87       184
   macro avg       0.86      0.86      0.86       184
weighted avg       0.87      0.87      0.87       184
```

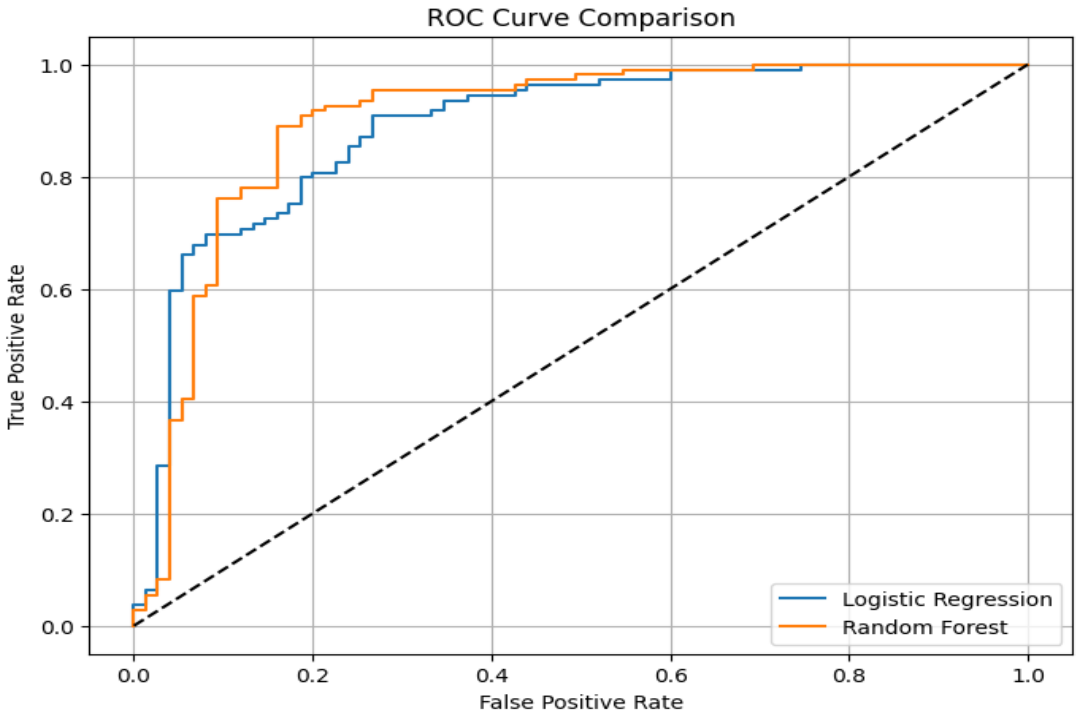
Difficulties:

- The **initial Random Forest model** achieved near **100% training accuracy**, which strongly indicated **overfitting**.
- Performance on the test set was significantly lower, revealing a **train-test performance gap**.

Overfitting Strategy:

- Compared training and test set performance (accuracy, AUC, F1)
- Tuned Random Forest parameters
- Evaluated via ROC curve and classification metrics

ROC curve comparison:



7. Analysis Results and Model Comparison:

Metric	Logistic Regression	Random Forest
Train Accuracy	82%	84%
Test Accuracy	80%	87%
Train AUC	0.89	0.92
Test AUC	0.88	0.90
Train F1 (1)	0.83	0.86
Test F1 (1)	0.83	0.89

Random Forest had a steeper ROC curve, confirming better true positive rate across thresholds.

8. Conclusion:

Based on the comparative analysis of both models, the **Random Forest classifier significantly outperformed Logistic Regression** across all major evaluation metrics, including accuracy, recall, F1-score, and AUC. While Logistic Regression provided a solid and interpretable baseline with an accuracy of 80% and an F1-score of 0.83, the Random Forest model achieved superior performance, reaching a test accuracy of **87%**, an F1-score of **0.89**, and an AUC of **0.90**.

Initially, Random Forest showed signs of overfitting with perfect training accuracy (100%), but after applying **hyperparameter tuning**—specifically limiting the maximum tree depth and setting a minimum number of samples per leaf—the model's generalization significantly improved. The updated model achieved a balanced trade-off between bias and variance, as reflected by the narrow gap between training and test performance.

From a real-world perspective, this makes the tuned Random Forest model highly suitable for **practical deployment in health screening systems**. Given its robust predictive power and ability to capture complex feature interactions, the model can effectively assist clinicians in identifying **high-risk patients** based on routinely collected clinical data such as age, cholesterol, resting blood pressure, and exercise-induced symptoms.

By leveraging this model, medical professionals can **prioritize preventive care**, make **data-informed diagnostic decisions**, and potentially **reduce the incidence of undiagnosed heart disease**. Overall, the results validate Random Forest as the most appropriate algorithm for this heart disease prediction task, combining accuracy, reliability, and practical utility.

REFERENCES:

1. Detrano, R., Janosi, A., Steinbrunn, W., Pfisterer, M., Schmid, J., Sandhu, S., Guppy, K., Lee, S., & Froelicher, V. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. American Journal of Cardiology, 64,304--310.

2. David W. Aha & Dennis Kibler. "Instance-based prediction of heart-disease presence with the Cleveland database."

3. Gennari, J.H., Langley, P, & Fisher, D. (1989). Models of incremental concept formation. Artificial Intelligence, 40, 11--61.